Introduction to Agents and Agentic Coding

Codable Meetup / 2026

Hello, I'm Tobi Omotayo

And I am passionate about Intelligent Autonomous Systems

Click to see full view

YOTAN Electric Can Crusher, 8oz 12oz 16oz Automatic Aluminum Can Crusher, Can Fill 8 Cans at a Time, Automatic Can Eject, Heavy Duty Can Crushers for Recycling Aluminum Seltzer, Cola, Beer Cans

Visit the YOTAN Store

4.5 ★★★★☆ (2) | Search this page

#1 New Release in Can Crushers

$104.99

Price history

Or $26.25 /2 weeks (x4). Select from 2 plans

Get $60 off instantly: Pay $44.99 upon approval for Amazon Visa. No annual fee.

FREE Returns

Save 10% on 2 select item(s) Shop items ›

Brand	YOTAN
Capacity	16 ounces
Color	BLACK3
Product Dimensions	3.9"D x 4.06"W x 17.76"H
Material	Metal

About this item

- ELECTRIC CAN CRUSHER FOR RECYCLING - Perates on standard household power—no air compressor needed. Upgraded motor and cooling system supports 20-minute continuous crushing

prime

Enjoy fast, free delivery, exclusive deals, and award-winning movies & TV shows.

Join Prime

$104.99

FREE delivery **Wednesday, March 18**

Or Prime members get FREE delivery **Monday, March 16.**

Join Prime

Deliver to Oluwatobi - Philadel... 19139

In Stock

Quantity: 1

Add to cart

Buy Now

Ships from	Amazon
Sold by	XZY-STORE
Returns	FREE 30-day refund/replacement
Payment	Secure transaction

See more

Agenda

What Are Agents?	10 min
Benchmarks	
Demo — Build a Coding Agent	8 min
Agentic Coding	
Impact of Agents in the SWE Lifecycle	8 min
Getting the Most from AI	7 min
Demo — Agentic Project Setup	12 min
Limitations & Workarounds	8 min
The Future of Agents	5 min

01

What Are AI Agents?

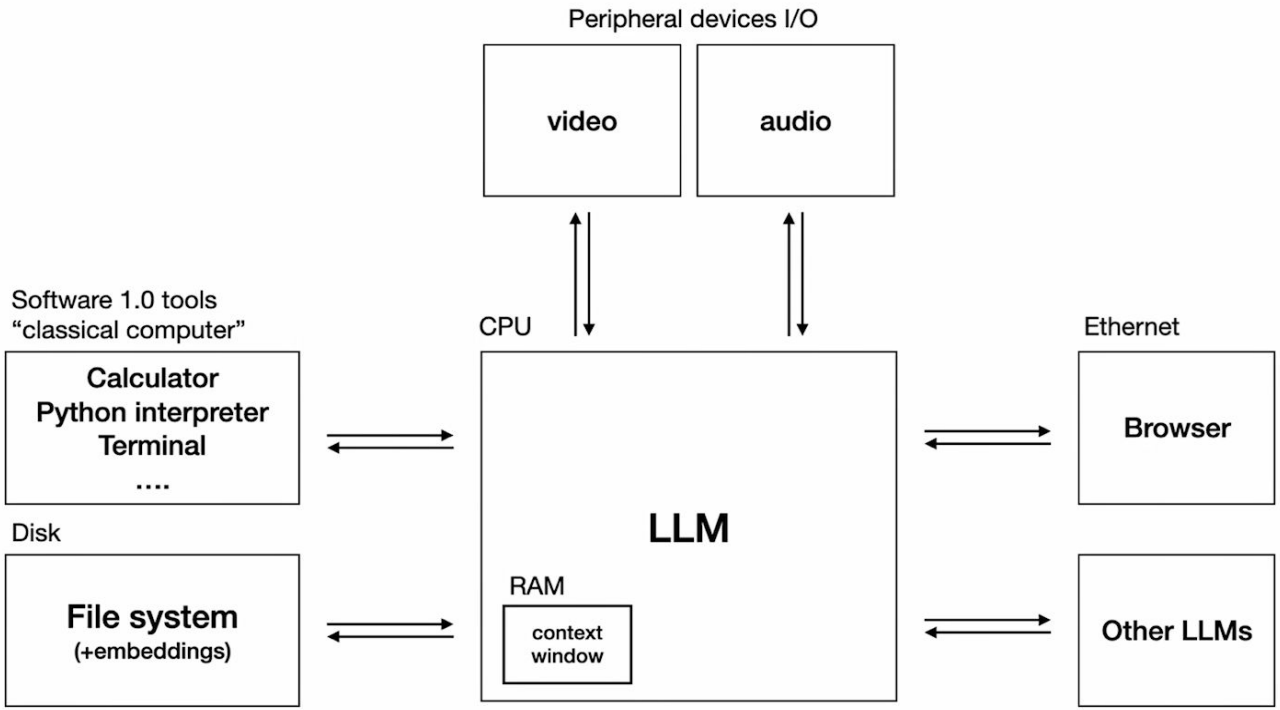
What is an agent?

- An Agent is a system that independently accomplish tasks on your behalf.

- It leverages an LLM to manage workflow execution and make decisions. It recognizes when a workflow is complete and can proactively correct its actions if needed. In case of failure, it can halt execution and transfer control back to the user.

- It has access to various tools to interact with external systems—both to gather context and to take actions—and dynamically selects the appropriate tools depending on the workflow’s current state, always operating within clearly defined guardrails

LLM OS


Peripheral devices I/O
video
audio
Software 1.0 tools
“classical computer”
Calculator
Python interpreter
Terminal
....
Disk
File system
(+embeddings)
CPU
LLM
RAM
context window
Ethernet
Browser
Other LLMs

LLM OS

- LLM is like the Central Processing Unit
- Tokens are the Bytes
- Prompts act like programs, and tools/APIs act like system calls
- Context window = RAM
- Engineering still matter: we design the system around the model, not just the prompt

How an Agent works?

Perceive → **Decide** → **Act** → **Iterate**

Read context, files, errors

Reason about next step

Execute tools, write code

Check results, refine

An agent = LLM + Tools + Autonomy + Feedback Loop

Tools

Tools make an agent powerful. The limit is available tools and context.

`Bash`

Shell commands

`Grep`

Search code

`LSP`

Language server

`LLDB`

Debugger

`npm`

Package mgmt

`xcodebuild`

Build system

The limit of any agent is its available tools and context.

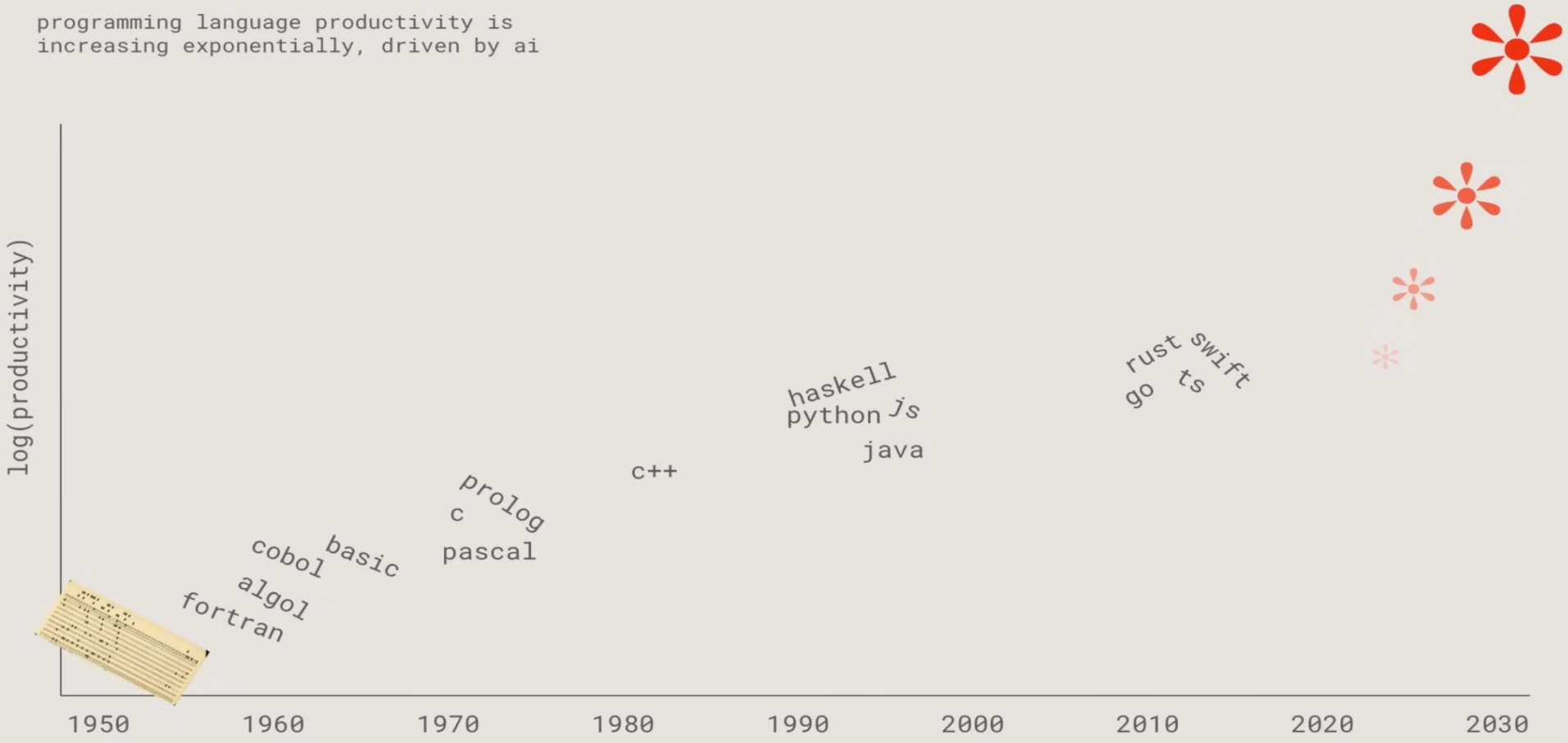
The levels of AGI

1	Chatbots	Conversational AI	
2	Reasoners	Solve novel problems	
3	**Agents**	Autonomous action	**← We are here**
4	Innovators	Aid in invention	
5	Organizations	Full org-level work	

02

Agentic Coding

programming language productivity is increasing exponentially, driven by ai



Software 1.0 vs Software 2.0

Software 1.0 (old style programming)

- Humans write the rules by hand in code (Python, C++, etc.).
- You think hard, then type:
  - “If this happens, do that.”
  - “To add two numbers, do X.”

Every behavior of the program comes from lines that a programmer explicitly wrote.

Software 2.0 (AI / neural networks)

- We don’t write all the rules ourselves.
- Instead, we:
  - Define a goal (e.g. “classify cat vs dog correctly”, “win the game”, “predict the next word”).
  - Collect a lot of data (pictures, text, examples, rewards).
  - Let an algorithm search for a program (a neural network) whose internal numbers (“weights”) make it good at that goal.

Software 2.0

- So, Software 2.0 automates things where you can clearly judge answers, even if you can’t write the rules.

- While conventional software enables users to streamline and automate workflows,

- Agents are able to perform the same workflows on the users’ behalf with a high degree of independence.

Demo 1

Show difference between Software 1.0 and Software 2.0 with XcodeBuildMCP testing an iOS App, one with XCUITest and the other with Just telling Codex to test what it just built - it uses accessibility APIs

Verifiability

- Key question in the era of Software 2.0
  - “Can we automatically check if the AI did a good job on each attempt?”

- Responsibility of Software 2.0 Engineers is shifting to verifying and reviewing code that AI has produced.

- Your job as a software engineer is to deliver code you have proven to work.
  - Doesn’t matter whether you wrote it manually, used voice input to edit the text or you instructed AI to do it for you.

- Because AI helps write code fast, people are writing more code than ever before and teams are now plagued with numerous PRs
  - Some have come up with ways to track which code was written by AI and which has been reviewed by a Human Engineer

AI Engineering

All Software Engineering will become AI Engineering

- Prompt Engineering
- Context Engineering
- Harness Engineering
- Evals
- Observability and Traceability
- Agent Orchestration

Prompt Engineering

This is how you give an agent instructions:

- Agents have System Prompt and User Prompt
  - Prompts can also come in different forms which we will see later.

- Different models require different prompting techniques

- Don’t assume the way you prompt a Claude Opus 4.6 will just work for GPT-5.4.You have to read the prompting guide for each model when it’s released.

Prompt Engineering

- For Software Engineering Tasks, treat prompts as executable specs, not just questions
  - Requirements: turn vague asks into clear user stories, constraints, and acceptance criteria
  - Design: generate options, trade-offs, edge cases, and architecture prompts
  - Build: scaffold code, refactor safely, and preserve patterns with repo-aware context
  - Test: create test cases, mocks, negative scenarios, and bug reproduction prompts
  - Review & Operate: support PR reviews, documentation, runbooks, and incident analysis
- What makes a good engineering prompt?
  - Context + Goal + Constraints + Examples + Output format + Verification

Demo

Show the audience some system prompt of common AI tools.

Evals and Benchmarks

- If you don’t measure it, you can’t trust it
- Benchmarks show general capability on standard tasks
- Evals test performance on your real workflows, codebase, and quality bar
- Measure what matters:
  - Correctness
  - Reliability
  - latency/cost
  - safety/compliance
- Good evals use representative tasks, clear pass/fail criteria, and repeatable scoring
- Use both: benchmarks for comparison, evals for deployment decisions

Rule of thumb:
Benchmarks tell you what a model can do. Evals tell you whether you should use it.

Popular Coding Benchmarks

What to pay attention to

SWE-Bench Verified	Real GitHub issues, human-verified. Gold standard.
SWE-Bench Pro	Harder multi-file, multi-step issues.
Terminal-Bench	Terminal tools, debugging, real dev environments.
HumanEval / MBPP	Basic code gen. Useful baseline, not the full picture.

Demo

Build your own coding agent

The best way to understand agents is to build one.

Building a coding agent

LIVE

01	**LLM API Integration**	Connect to the model
02	**Tool Definitions**	Define what the agent can do
03	**The Agent Loop**	Perceive → Decide → Act → Iterate
04	**Error Handling**	Recovery and self-correction

Impact of agents in the software engineering lifecycle

The case for agentic coding

Automate the boring parts	Boilerplate, config, repetitive refactoring
Democratise expertise	Don't need to master every framework
Research, don't guess	No more StackOverflow trial-and-error
Experiment faster	Try approaches, compare metrics and trade-offs
Multi-system integration	One agent connects to all your tools

Domain knowledge is your leverage

Your domain expertise × AI = maximum output

If you lack domain knowledge:

1. Ask for pros & cons with references
2. Read the references yourself
3. Check comments and rebuttals
4. Agents are opinionated — push back

Demo

Setting up your project for agentic coding
Setup Prompt and Context Engineering: agents.md file
Add Agent Skills

It doesn't work by just expecting it to work.

Give the agent eyes

Error output	Test results	Build logs	UI screenshots	Architecture docs

Most failures happen because the agent lacks context — it can't see the result of its actions.

Demo project ideas

Interview Prep App

Help engineers prepare for tech interviews

Remote Jobs Finder

Find remote roles hiring Nigerians, paid in USD

Nigeria Economy Dashboard

Interactive charts, indicators, state-level breakdown

Food Species Identifier

Identify Nigerian food breeds, allergens, sourcing

03

Limitations & Workarounds

Limitations

Hallucinations

Confidently generates wrong information. Always verify critical outputs.

Lack of context

Doesn't know your codebase unless you tell it. Provide docs, logs, structure.

Sycophancy

"You're absolutely right!" — agrees even when you're wrong. Push back.

More limitations

Token costs

Tokens are expensive — budget and optimize.

Context window

Quality degrades with length. Start fresh sessions when needed.

SOTA chasing

Build great workflows, not model-dependent pipelines.

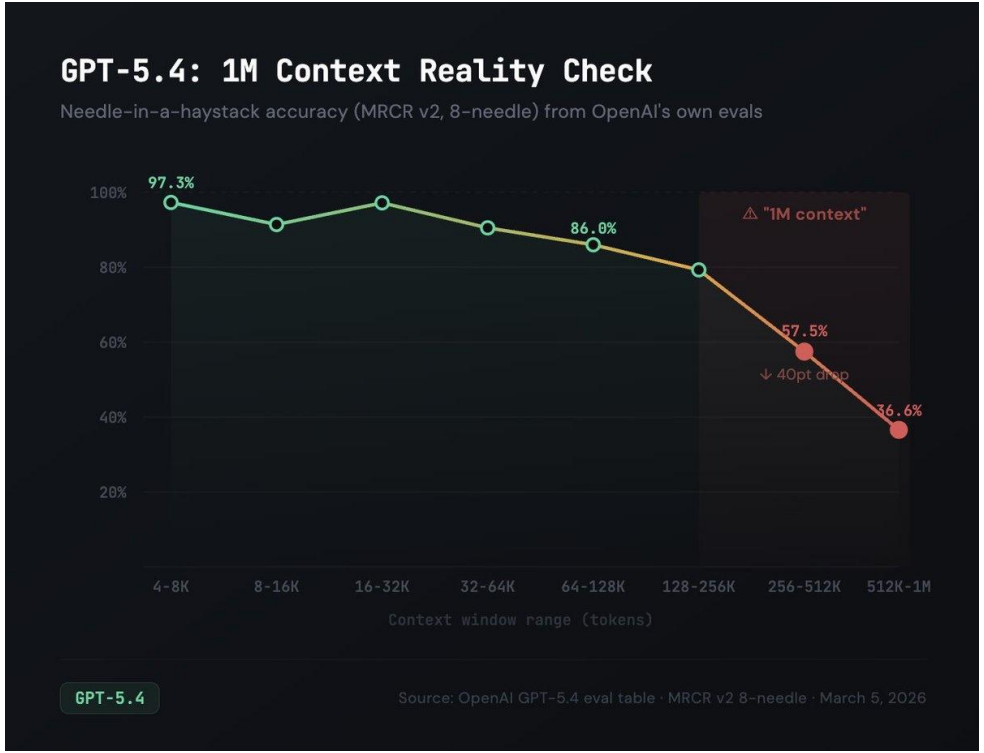
Multi-turn drift

LLMs lose coherence in long conversations. Keep sessions focused.

Develop internal evals to compare models — your chart should trend upward over time.

Context rot

MRCR v2 — needle-in-a-haystack: how well can a model retrieve information hidden in vast text?



The future of agents

Humans = the bottleneck

Agents can work 24/7. We can't.

Dedicated role agents

Autonomous, proactive, budget-aware.

Background agents

No UI — just reports and status updates.

Reliability is key

None of this works without trust.

Questions
&
Response

Thank you.

Join the AI Engineering WhatsApp Group
[QR Code Here]

AI Engineering WhatsApp Group

Codable Meetup GitHub Org